



SAPIENZA
UNIVERSITÀ DI ROMA

Impossibility of Automated Theorem Proving for Propositional Proof Systems by Analyzing Meta- Complexity Statements

Faculty of Information Engineering, Computer Science and Statistics
Master's Degree in Computer Science

Simone Bianco

ID number 1986936

Advisor

Prof. Nicola Galesi

Prof. Massimo Lauria

Academic Year 2025/2026

**Impossibility of Automated Theorem Proving for Propositional Proof Systems
by Analyzing Meta-Complexity Statements**

Master's Thesis. Sapienza University of Rome

© 2026 Simone Bianco. All rights reserved

This thesis has been typeset by $\text{\LaTeX}$ and the Sapthesis class.

Author's email: bianco.simone@outlook.it

something said by someone smarter than me
— not me

Abstract

Contents

1	Introduction	1
1.1	Historical background	1
2	Preliminaries	3
2.1	Propositional logic	3
2.2	Encodings of combinatorial statements (TODO)	4
2.3	Computability	4
2.4	Computational complexity	6
3	Proof complexity	8
3.1	Propositional proof systems	8
3.2	Common proof systems	9
3.2.1	Resolution	9
3.2.2	Frege systems	11
3.2.3	Algebraic systems	12
3.2.4	Optimization-based systems	14
3.2.5	Tree-like systems	14
3.3	Simulations between proof systems	16
3.4	Complexity measures for proof systems	17
3.5	Game characterizations of Resolution measures	19
3.5.1	The Prover-Delayer Game (TODO)	19
3.5.2	The Spoiler-Duplicator Game (TODO)	19
4	Automatability	20
5	Analyzability	21
	Conclusions	22
	Acknowledgements	23

Chapter 1

Introduction

1.1 Historical background

It's 1956. The genius mathematician John von Neumann is hospitalized at the Walter Reed Army Medical Hospital in Washington, USA. One year earlier, a mass was found near his collarbone, which turned out to be cancer, probably caused by exposure to radiation at the Los Alamos National Laboratory during World War II [Pou92]. After his condition improved thanks to a successful treatment, his colleague and friend Kurt Gödel sent a letter to express solidarity for his diagnosis, hoping for a full recovery. To keep von Neumann busy during his hospital stay, Gödel proposed his friend a seminal problem of fundamental interest both for logic and computer science.

maybe move to automatability chapter

«One can obviously easily construct a Turing machine, which for every formula F in first order predicate logic and every natural number n , allows one to decide if there is a proof of F of length n (length = number of symbols). Let $\psi(F, n)$ be the number of steps the machine requires for this and let $\varphi(n) = \max_F \psi(F, n)$. The question is how fast $\varphi(n)$ grows for an optimal machine. One can show that $\varphi(n) \geq kn$.

If there really were a machine with $\varphi(n) \sim kn$ (or even $\sim kn^2$), this would have consequences of the greatest importance. Namely, it would obviously mean that in spite of the undecidability of the Entscheidungsproblem, the mental work of a mathematician concerning Yes-or-No questions could be completely replaced by a machine. After all, one would simply have to choose the natural number n so large that when the machine does not deliver a result, it makes no sense to think more about the problem.»

Gödel to von Neumann, Princeton, 20 March 1956 [Sip92]

In a less formal way, Gödel's question asks if there is an efficient algorithm capable of deciding if a given mathematical statement has a proof of some given length. In modern times, the above problem became known as the *Proof Size Problem* (PSP). Sadly, an answer to Gödel's letter was never found and is supposed to have never existed. In fact, during his long hospital stay, von Neumann's mental health

rapidly declined due to his extreme thanatophobia, the fear of impending death. Almost a year later after being hospitalized, von Neumann died [Pou92].

In retrospective, Gödel's question is an historic precursor of the P *vs.* NP question. The original formulation of this latter question asks whether problems whose solutions are easy to double-check, i.e. problems who lay in the class NP, are also intrinsically easy to solve from scratch, i.e. if they also lay in the class P. The most interesting problems of these classes are the so called NP-complete problems, that being problems in NP whose efficient solution would imply an efficient solution for all NP problems, i.e. $P = NP$. The current state of the art conjectures that $P \neq NP$, meaning that these so-called complete problems cannot be efficiently solved in their general case.

If this were the case, there would be no better algorithmic approach than that of *brute force*, an approach that tests every candidate solution in the search space defined by the problem itself. For instance, if a problem asks to determine if there is a sub-structure with a specific property, the brute force approach tests every possible sub-structure. In his letter, Gödel himself states that he sees no better approach than that of reducing the search space through clever techniques.

During the years, researchers began to explore the Proof Size Problem from a restricted point of view. In particular they tried to prove or disprove the possibility of efficiently solving the problem for specific *proof systems*, i.e. logical systems that use restricted rules to find proofs of statements. For many weak proof systems it has been proven that the answer is negative, but it remains open whether the result may hold for more powerful proof systems.

add citation from
Kazuo Iwama

A more general question related to proof search and proof systems is that of *automatability*. Automatability asks if for a fixed proof system Q one can find an efficient algorithm that returns a Q -proof of a given statement, assuming it exists. Differently from the Proof Size Problem, which basically asks if short proof exists, automatability asks if short proofs can be constructed. The automatability of a proof system gives a way to approximate the minimum proof-size of a formula up to a polynomial factor (see).

add section on why
this is true

A central discovery of modern proof complexity is that many natural proof systems are not automatable under some valid complexity assumption (e.g. standard cryptographic assumptions or even $P \neq NP$). These results highlight a certain controversy in the world of proof systems. One would expect that a more expressive proof system would be capable of achieving remarkably short proofs, making it easier to generate proofs. However, the very same increase in expressive power implies that the search space of those proofs may be so unstructured that no algorithm can find them faster than brute force. In other words, it seems to be that if an algorithm fails to discover a short proof that is guaranteed to exist then the fault lies not in the algorithm, but in the proof space itself.

Add paragraph on
analyzability

Chapter 2

Preliminaries

2.1 Propositional logic

Propositional logic serves as the foundational mathematical framework for reasoning over declarative statements using boolean truth values and formal logical operators. For our purposes, we'll be interested in a fragment of propositional logic that works with finite amount of elements.

Let $\mathcal{P} = \{x_1, \dots, x_n\}$ denote a non-empty, finite set of atomic propositional symbols, called *variables*. Let $\Omega = \{\wedge, \vee, \neg, \rightarrow, \leftrightarrow\}$ be a set of logical connectives. Each of these constructors represents, respectively, the concept of conjunction, disjunction, negation, implication and logical equivalence.

The set of formulas $\text{Fml}(\mathcal{P})$ generated by $\mathcal{P}$ is the free algebra generated by $\mathcal{P}$. Inductively, $\text{Fml}(\mathcal{P})$ is the smallest set satisfying the following three conditions:

- $\mathcal{P} \subset \text{Fml}(\mathcal{P})$;
- If $\varphi \in \text{Fml}(\mathcal{P})$ then $\neg\varphi \in \text{Fml}(\mathcal{P})$;
- If $\varphi, \psi \in \text{Fml}(\mathcal{P})$ and $\circ \in \Omega$ then $(\varphi \circ \psi) \in \text{Fml}(\mathcal{P})$.

The set $\text{Fml}(\mathcal{P})$ defines propositional logic from a syntactical point of view, i.e. how correct formulas are written. To give meaning to the formulas, we have to also define it from a semantical prospective. An *assignment* is a mapping $\alpha : \mathcal{P} \rightarrow \{0, 1\}$ that sets a truth value to each variable. By extension, each assignment α over variables induces an assignment $\hat{\alpha} : \text{Fml}(\mathcal{P}) \rightarrow \{0, 1\}$ through the semantical meaning of the logical connectives:

- $\hat{\alpha}(\neg\varphi) = 1$ if and only if $\hat{\alpha}(\varphi) = 0$
- $\hat{\alpha}(\varphi \wedge \psi) = 1$ if and only if $\hat{\alpha}(\varphi) = \hat{\alpha}(\psi) = 1$
- $\hat{\alpha}(\varphi \vee \psi) = 0$ if and only if $\hat{\alpha}(\varphi) = \hat{\alpha}(\psi) = 0$
- $\hat{\alpha}(\varphi \rightarrow \psi) = 1$ if and only if $\hat{\alpha}(\psi) = 1$ whenever $\hat{\alpha}(\varphi) = 1$ holds
- $\hat{\alpha}(\varphi \leftrightarrow \psi) = 1$ if and only if $\hat{\alpha}(\varphi \rightarrow \psi) = \hat{\alpha}(\psi \rightarrow \varphi) = 1$

φ	ψ	$\neg\varphi$	$\varphi \wedge \psi$	$\varphi \vee \psi$	$\varphi \rightarrow \psi$	$\varphi \leftrightarrow \psi$
1	1	0	1	1	1	1
1	0	0	0	1	0	0
0	1	1	0	1	1	0
0	0	1	0	0	1	1

Table 2.1. Summary of truth assignments for logical connectives in Ω .

By recursively combining these constructs, we can build more complex formulas that fully express a sentence. For instance, the sentence “I’ll take my umbrella and my boots if it rains” can be modeled by the formula $R \rightarrow (U \wedge B)$. A formula $\varphi \in \text{Fml}(\mathcal{P})$ is said to be *satisfiable* if there is at least one assignment α such that $\alpha(\varphi) = 1$. When all possible assignments satisfy a formula, we refer to that formula as a *tautology*, or simply as true formulas. Clearly, φ is a tautology if and only if $\neg\varphi$ is unsatisfiable.

Given a Boolean variable x , a *literal* ℓ of x is either the variable x_i itself or its negation $\neg x$. A *clause* is a disjunction of literals, $C = \ell_1 \vee \dots \vee \ell_k$, while a *term* is a conjunction of them, $T = \ell_1 \wedge \dots \wedge \ell_t$. A *conjunctive normal form* (CNF) formula φ is a conjunction of m clauses $C_1, \dots, C_m$. Similarly, a *disjunctive normal form* (DNF) formula is a disjunction of s terms $T_1, \dots, T_s$. Any formula can be written as an equivalent CNF or an equivalent DNF. For instance, the formula $\varphi = ((x_1 \vee \neg x_2) \wedge x_3) \vee x_4$ can be written as the CNF formula $(x_1 \vee \neg x_2 \vee x_4) \wedge (x_3 \vee x_4)$ or as the DNF formula $(x_1 \wedge x_3) \vee (\neg x_2 \wedge x_3) \vee x_4$.

2.2 Encodings of combinatorial statements (TODO)

2.3 Computability

Depending on the objective, a problem can ask for a binary answer, a specific solution structure or the best possible solution out of many. We divide these problems into three categories: *decision*, *search* and *optimization*. A decision problem is a problem for which we’re asked to determine if a chosen property holds for a given an instance of the problem. A search problem, instead, asks to find some structure that certifies the validity of the chosen property. Similarly, an optimization problem asks us to find the optimal structure among all the available ones. It’s easy to see that these three ideas form some sort of hierarchy between types of problems. In general, optimization problems are harder than search problems, which in turn are harder than decision problems.

We face these types of problems every day in our life. For instance, suppose that we want to reach a destination B from a starting point A . If we’re interested in finding the *shortest* route between point A and B , this becomes an optimization problem. If we want find any kind of route between the two sites, without caring about its length, this becomes a search problem. If, instead, we want to just establish

the existence of at least one route, this becomes a decision problem.

Let Σ be a finite set of symbols (e.g. $\Sigma = \{a, b, c, 0, \$, 3, x, \dots\}$), referred to as *alphabet*, and let Σ^* be the infinite set of all finite strings formed by characters taken from that alphabet. Any subset $L \subseteq \Sigma^*$ is referred to as a *language*. Typically, we're interested to the case where $\Sigma = \{0, 1\}$.

Consider a binary predicate $P : \Sigma^* \rightarrow \{0, 1\}$ representing some property, meaning that $P(x) = 1$ means that the property holds for the input instance x and $P(x) = 0$ means that it doesn't. Let $R_P \subseteq \Sigma^* \times \Sigma^*$ be the binary relation such that $R_P(x, y)$ holds if and only if y certifies the fact that $P(x) = 1$. We refer to $\text{Sol}_P(x) = \{y \in \Sigma^* \mid R_P(x, y)\}$ as the set of *solutions* (or *witnesses*) for the instance x over the property P .

Definition 2.1. The decision problem of P is defined as the language $\Pi_{\text{dec}}^P = \{x \in \Sigma^* \mid \text{Sol}_P(x) \neq \emptyset\}$. The problem Π_{dec}^P is said to be computable if there is an algorithm $\mathcal{A}$ such that $\mathcal{A}(x) = 1$ when $\text{Sol}_P(x) \neq \emptyset$ and $\mathcal{A}(x) = 0$ otherwise.

We observe that the above definition of computability for decision problem can also be stated as the existence of an always-halting algorithm $L(\mathcal{A}) = \Pi_{\text{dec}}^P$. Moreover, we also observe that the above definition could've been stated without the use of the binary relationship R_P since $\Pi_{\text{dec}}^P = \{x \in \Sigma^* \mid P(x) = 1\}$ is an equivalent definition. In this setting, the definition is given using R_P only for consistency with search and optimization problems. In fact, for the other two types of problems binary relationships are necessary due to the fact that one instance may have more than one possible witness. For instance, there may be more than one (optimal) route from point A to point B .

Definition 2.2. The search problem of P is defined as the binary relationship $\Pi_{\text{search}}^P = R_P$. The problem Π_{search}^P is said to be computable if there is an algorithm $\mathcal{A}$ such that $\mathcal{A}(x) \in \text{Sol}_P(x)$ when $\text{Sol}_P(x) \neq \emptyset$ and $\mathcal{A}(x) = \perp$ otherwise.

Here, the symbol $\perp$ is used to represent the fact that no solution exists for the given input instance. We observe that the definition allows any algorithm computing the problem to return a different solution on each execution. Optimization problems require an additional component: a *cost function* $c : \Sigma^* \times \Sigma^* \rightarrow \mathbb{R}$ expressing the optimality of the solution for the given instance. We denote the set of optimal solutions as $\text{Opt}_P(x) = \{y \in \text{Sol}_P(x) \mid c(x, y) = \max_{z \in \text{Sol}_P(x)} c(x, z)\}$.

Definition 2.3. The optimization problem of P over c is defined as the binary relationship $\Pi_{\text{opt}}^P = \{(x, y^*) \in \Sigma^* \times \Sigma^* \mid y^* \in \text{Opt}_P(x)\}$. The problem Π_{opt}^P is said to be computable if there is an algorithm $\mathcal{A}$ such that $\mathcal{A}(x) \in \text{Opt}_P(x)$ when $\text{Sol}_P(x) \neq \emptyset$ and $\mathcal{A}(x) = \perp$ otherwise.

As previously stated, these three categories form an hierarchy of problems. This comes clear from their own definitions: any algorithm computing an optimization problem also computes a search problem and any algorithm computing a search problem can be adapted into one that computes a decision problem.

2.4 Computational complexity

In his seminal work, Turing [Tur37] proved that some problems are *undecidable*, meaning that there is no algorithm that can compute them exactly. Following his work, the pioneers of computer science started noticing that even though many problems can be computed, the algorithms computing them would take too much resources, making them infeasible to process for large inputs. In particular, they focused on two main measures: *time* and *space*.

Definition 2.4. Let $\mathcal{A}$ be an algorithm and let $t, s: \mathbb{N} \rightarrow \mathbb{R}^+$ be two functions. We say that $\mathcal{A}$ *runs in time* t if on every input x the algorithm halts after at most $t(|x|)$ steps and that $\mathcal{A}$ *runs in space* s if on every input x it uses at most $s(|x|)$ cells memory cells.

These two complexity measures are intrinsically related one to another and they are, usually, negatively correlated. Generally, if we allow our algorithm to use more space then the computation can be sped up by saving partial computations, while if we want to lower the amount of space needed then these partial computation have to be computed each time they're required.. Larger inputs require a larger amount of computational resources, making the values $t(n)$ and $s(n)$ proportional to the size n of the input. For this reason, as the input size grows, we are interested in the asymptotic behavior of these measures.

To distinguish between efficiently solvable problems and intractable ones, we usually consider the class P . This class is defined as the set of all decision problems solvable in polynomial time by an algorithm, meaning that there is an algorithm that computer the problem in time $O(n^c)$, for some $c > 0$. Similarly, we consider the class FP as the class of search problems solvable in polynomial time by an algorithm.

We observe that this definition allows the exponent c to be as huge as possible. In real world situations, any algorithm with runtime larger than $O(n^3)$ is considered useless since it requires too much time even for smaller inputs. Nonetheless, this can be used at our advantage: if a problem doesn't lie in P , it means that there is no algorithm for it even if we were to consider infinitely large values of c . Therefore, a decision problem L is considered efficiently solvable when it lies in P , while it is considered intractable when it doesn't. The same reasoning applies to the class FP .

The aforementioned P vs. NP (or the FP vs. FNP question for search problems) question asks to settle if the class of efficiently solvable problems corresponds to the class of efficiently verifiable problems. Here, a *verifier* for a problem L is an algorithm that takes an additional input w , referred to as the *witness*, and uses it to check if $x \in L$ holds or not. Usually, this witness is a string that describes a real solution of the problem for the given input. By definition, we have that $P \subseteq NP$ since if a problem is efficiently solvable then we can just ignore the given witness and solve the problem. Hence, the P vs. NP question asks to settle if this inclusion also holds in the converse direction, i.e. $NP \subseteq P$. After almost 60 years of scientific literature studying this problem and its ramifications, researchers conjecture that the answer to this problem is negative.

Conjecture 2.1. $P \neq NP$

For our purposes, we'll assume this conjecture to be true. Another class of our interest is the class coNP , defined as the set of all problems whose complement lies in NP , i.e. $\text{coNP} = \{L \subseteq \Sigma^* \mid \bar{L} \in \text{NP}\}$. It's easy to see that problems that lie in coNP admit a verifier that use witnesses to discard invalid input instances: if w is a witness for the membership of x in $\bar{L}$ then w is also a witness for the non-membership of x in L . In other words, a problem is in NP if its correct instances can be efficiently verified, while a problem is in coNP if the same holds for incorrect instances.

Again, it's easy to see that $\text{P} \subseteq \text{coNP}$, but the converse is not known and it is actually a question equivalent to establishing that $\text{P} \subseteq \text{NP}$. This is due to the fact that if a problem is efficiently solvable then its complement is also efficiently solvable (we can just negate the output of the same algorithm). This same reasoning implies that if $\text{P} = \text{NP}$ then $\text{NP} = \text{coNP}$. Therefore, proving that $\text{NP} \neq \text{coNP}$ could be a way of establishing that $\text{P} \neq \text{NP}$. It is also conjectured that $\text{NP} \neq \text{coNP}$ since there are evident asymmetries between the hardness of witnessing that something holds and that of witnessing something everywhere in mathematics and computer science.

Conjecture 2.2. $\text{NP} \neq \text{coNP}$

Finally, we introduce the concept of reductions between problems. Let A and B be two decision problems. We say that A *many-to-one reduces* to B , written as $A \leq_m B$ when there is a computable function $f : \Sigma^* \rightarrow \Sigma^*$ such that $x \in A$ if and only if $f(x) \in B$. The idea of problem reduction is simple: if a reducing function exists and the new problem can be solved then the original problem can also be solved. This is due to the fact that if an algorithm for the original problem has to simply compute the reduction and then execute the algorithm that solves the new problem.

It's easy to see that the relation $\leq_m$ forms a partial order over the power set $2^{\Sigma^*} = \{S \subseteq \Sigma^*\}$, corresponding to the set of all decision problems. When $A \leq_m B$ holds, we say that B is “at least as hard as” A since solving B implies solving A . If the reduction is computable in polynomial time, we say that A *p-reduces* to B , written as $A \leq_p B$. In two seminal papers, Cook and Levin independently showed that every problem that lies in NP is reducible to the *satisfiability problem* $\text{SAT} = \{\langle \varphi \rangle \mid \varphi \text{ a satisfiable CNF}\}$, where $\langle \cdot \rangle$ is the binary encoding. Problems with this property are referred to as NP-hard and, under the assumption that $\text{P} \neq \text{NP}$, they cannot be solved in polynomial time. Therefore, showing the hardness of a problem comes as a tool of showing that it cannot be efficiently solved.

[add reference](#)

Chapter 3

Proof complexity

3.1 Propositional proof systems

Proof complexity is the branch of computational complexity and logic that studies the resources that are needed in order to *certify* mathematical statements, rather than of the resources needed to *decide* them. At its core, proof complexity asks if there is a way of writing down proofs of propositional tautologies so that every tautology has short proofs. Such ways of proof writing can be viewed as sets of rules capable of manipulating propositional formulas, deriving new formulas from them. These sets of rules are referred to as *propositional proof systems*. When a formula ψ can be derived from a formula φ through the rules of a proof system Q , we write $\varphi \stackrel{Q}{\vdash} \psi$ or simply $\varphi \vdash \psi$ when the context is clear. The easiest example of propositional proof system is *Resolution*, based on two single rules:

$$\frac{C \vee x \quad D \vee \neg x}{C \vee D} \quad (\text{resolution}) \qquad \frac{C}{C \vee B} \quad (\text{weakening})$$

For a more detailed discussion on examples of proof systems, see the next section .  In the seminal paper from which proof complexity originated, defined proof systems as surjective functions $Q : \Sigma^* \rightarrow \text{TAUT}$, where the latter is the set of all tautologies, $\text{TAUT} = \{\langle \varphi \rangle \mid \varphi \text{ a CNF tautology}\}$. The idea behind this definition is that of an algorithm that is able to determine which tautology is proven by a given proof. A modern and more descriptive definition can be achieved through the concept of verifier.

Definition 3.1. A propositional proof system (p.p.s.) is a verifier Q such that for all formulas φ it holds that $\varphi \in \text{TAUT}$ if and only if there is a proof $\pi \in \Sigma^*$ such that $Q(\varphi, \pi) = 1$ and Q runs in time $O((|\varphi| + |\pi|)^c)$ for some $c > 0$.

This if and only if definition can be split in two implications called completeness and soundness. A proof system Q is *complete* if whenever a formula has a Q -proof it also is a tautology. Vice versa, Q is *sound* if whenever a formula is a tautology it also has a Q -proof. When π is a Q -proof of φ , we often write $\pi : Q \vdash \varphi$.

When for each tautology there is a short proof, i.e. $|\pi| = O(|\varphi|^c)$ for some $c > 0$, we say that the proof system is *polynomially bounded*. Proof systems were originally

introduced by Cook and Reckhow as a possible way of attacking the NP vs. coNP problem. In fact, through the modern definition, it's immediate to see that finding a polynomially bounded p.p.s. is equivalent to proving that $\text{NP} = \text{coNP}$.

Proposition 3.1. *There is a polynomially bounded p.p.s. if and only if $\text{NP} = \text{coNP}$*

Proof. One can easily prove that TAUT is coNP-complete. For our purposes, we can omit this proof and assume it to hold. Let Q be a polynomially bounded p.p.s. Then, since for each formula there is a proof of size $|\pi| = O(|\varphi|^c)$ for some $c > 0$, Q is a verifier for TAUT with polynomial runtime with respect to the input size $|\varphi|$, concluding that $\text{TAUT} \in \text{NP}$. By completeness, we get that $\text{NP} = \text{coNP}$.

Vice versa, suppose that $\text{NP} = \text{coNP}$. Then, since $\text{TAUT} \in \text{NP}$, there is a polynomial time verifier V for TAUT. In order for it to run in polynomial time, for each formula φ there must be at least one witness t such that $|w| = O(|\varphi|^d)$ for some $d > 0$, otherwise reading the witness would always require super-polynomial runtime, raising a contradiction. This concludes that V is a polynomially bounded p.p.s. $\square$

In their programme, Cook and Reckhow outlined a possible roadmap to proving that $\text{NP} \neq \text{coNP}$ through p.p.s. Their idea involved proving that strong proof systems require super-polynomial size proofs for some formulas, to then generalize the various results to any possible proof system, i.e. showing that every proof system has at least one formula that is hard to prove. However, researchers quickly realized that this task would be tedious even for weaker systems. In fact, even after almost 40 years of research in the field, only a few weak systems have been completely outlined (e.g. Resolution).

The programme has also a second, entirely practical, motivation. Every complete algorithm that established the satisfiability of a propositional formula, referred to as a *SAT-solver*, implicitly produces a proof of that formula in some proof system. For instance, the DPLL algorithm (Davis-Putnam-Logemann-Loveland) and CDCL solvers (Conflict-driven Clause Learning) produce resolution proofs, while Gröbner basis solvers produce algebraic proofs. A lower bound on proof size in a system is therefore an unconditional lower bound on the running time of *every* algorithm of the corresponding kind, regardless of how cleverly it is implemented. Proof complexity is, from this point of view, the theory of the limits of automated reasoning.

3.2 Common proof systems

3.2.1 Resolution

The Resolution proof system (Res), introduced by Robinson, is the most extensively studied proof system, extensively in automated theorem proving, SAT-solvers and artificial intelligence. Like many weak proof systems, Resolution's proofs are actually *refutations*, i.e. "proofs" of unsatisfiability of a formula. This is a feature, not an issue: since φ is a tautology if and only if $\neg\varphi$ is unsatisfiable, any refutation of $\neg\varphi$ is actually a proof of φ . In many works, the word refutation is often used as a synonym of proof. To help the reader, this won't be the case in this work.

add ref

$D_1 :$	$\neg x_3$	Axiom
$D_2 :$	$\neg x_1 \vee \neg x_2 \vee x_3$	Axiom
$D_3 :$	$x_1 \vee \neg x_2 \vee x_3$	Axiom
$D_4 :$	$\neg x_1 \vee \neg x_2$	$\text{Res}(D_1, D_2)$
$D_5 :$	$x_1 \vee \neg x_2$	$\text{Res}(D_1, D_3)$
$D_6 :$	$x_2 \vee x_3$	Axiom
$D_7 :$	$x_2 \vee \neg x_3$	Axiom
$D_8 :$	x_2	$\text{Res}(D_6, D_7)$
$D_9 :$	$\neg x_1$	$\text{Res}(D_4, D_8)$
$D_{10} :$	x_1	$\text{Res}(D_5, D_8)$
$D_{11} :$	$\perp$	$\text{Res}(D_9, D_{10})$

Figure 1. Example of a Resolution refutation for the formula $(x_2 \vee x_3) \wedge (x_2 \vee \neg x_3) \wedge (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_2 \vee x_3) \wedge \neg x_3$

Resolution refutations are based on repeated applications of two simple rules, starting from the axioms of the initial formula:

$$\frac{C \vee x \quad D \vee \neg x}{C \vee D} \quad (\text{resolution}) \qquad \frac{C}{C \vee B} \quad (\text{weakening})$$

Given a CNF formula $\varphi = C_1 \wedge \dots \wedge C_m$, let $\text{Ax}(\varphi) = \{C_1, \dots, C_m\}$ be the set of axioms of φ . Then, a Resolution refutation of φ is a sequence of clauses $\pi = (D_1, D_2, \dots, D_t)$ such that for each $i \in [t]$ exactly one of the following holds:

- *Axiom:* $D_i \in \text{Ax}(\varphi)$;
- *Resolution:* $D_i = \text{Res}(D_j, D_k)$ with $j, k \in [i-1]$;
- *Weakening:* $D_i = \text{Weak}(D_j)$ with $j \in [i-1]$.

For the last clause D_t , it must hold that $D_t = \perp$, where $\perp$ represents the *empty clause*, i.e. the clause that doesn't contain any literal.

The idea behind the Resolution proof system comes from the propagation of satisfiability underlying the resolution rule and the weakening rule.

Proposition 3.2. *Let α be an assignment. If $\alpha(C \vee x) = 1$ and $\alpha(D \vee \neg x) = 1$ then $\alpha(C \vee D) = 1$. Similarly, if $\alpha(C) = 1$ then $\alpha(C \vee B) = 1$*

One can easily prove by induction that if a clause Z is derived through repeated applications of resolution and weakening starting from the axioms then $\alpha(\varphi) = 1$ implies $\alpha(Z) = 1$. When $Z = \perp$, we reach a contradiction since, by definition, the empty clause is unsatisfiable. Therefore, if we're able to derive $\perp$ starting from $\text{Ax}(\varphi)$, we obtain a valid proof of the fact that φ must be unsatisfiable. This proves the soundness of Resolution and completeness can be proven in a similar way.

For any fixed natural number $k \in \mathbb{N}$, we define an extension of the Resolution proof system called *Resolution over k -DNFs*, denoted $\text{Res}(k)$, first introduced by . [add ref](#)

This proof system works over sets of k -DNF formulas, expressed as a disjunction of

conjunctions of at most k literals. Its rules are inspired by those of Resolution, adapting them to manage k -DNFs:

$$\begin{array}{c}
 \frac{\mathcal{D}_1 \vee \left(\bigvee_{j \in J} \ell_j \right) \quad \mathcal{D}_2 \vee \left(\bigwedge_{j \in J} \neg \ell_j \right)}{\mathcal{D}_1 \vee \mathcal{D}_2} \quad (\text{cut}) \\
 \\
 \frac{\mathcal{D}_1 \vee \left(\bigwedge_{j \in J} \ell_j \right) \quad \mathcal{D}_2 \vee \left(\bigwedge_{j' \in J'} \ell_{j'} \right)}{\mathcal{D}_1 \vee \mathcal{D}_2 \vee \left(\bigvee_{j \in J \cup J'} \ell_j \right)} \quad (\wedge \text{ introduction}) \\
 \\
 \frac{\mathcal{D}_1}{\mathcal{D}_1 \vee \left(\bigwedge_{j \in J} \ell_j \right)} \quad \frac{\mathcal{D} \vee \left(\bigwedge_{j \in J \cup J'} \ell_j \right)}{\mathcal{D}_1 \vee \left(\bigvee_{j \in J} \ell_j \right)} \quad (\text{weakenings})
 \end{array}$$

where $\mathcal{D}_1$ and $\mathcal{D}_2$ are k -DNF formulas.

3.2.2 Frege systems

Frege systems are a more classical type of propositional proof system, whose proofs are intended to infer tautologies. These proof systems are well known in the classical Logic literature and, furthermore, they work not only with CNF formulas, but also with general Boolean formulas.

Formally, a Frege system is a propositional proof system built from a finite set of axiom schemes and a finite set of rules of inference that can be applied to sets of formulas. An *axiom scheme* is a meta-formula with variables over formulas that can be substituted by any formula. For instance, both $x_1 \wedge x_2 \rightarrow x - 1$ and $(x_1 \vee x_2) \wedge x_2 \rightarrow (x_1 \vee x_2)$ are formulas that can be obtained from the axiom schema $(P \wedge Q) \rightarrow Q$, where P and Q are variables over $\text{Fml}(\mathcal{P})$. A *rule scheme* is a typical rule of logical derivation. A prominent example of a schema rule is the *Modus Ponens* (MP), defined as:

$$\frac{P \quad P \rightarrow Q}{Q}$$

Modus Ponens allows to cut implications from formulas of the form $P \rightarrow Q$ once the formula P alone is derived. We can define different Frege systems introducing different axiom schemes. Nonetheless, Reckhow proved that all Frege systems are polynomially equivalent to each other, meaning that the axiom scheme and rule scheme chosen is not relevant, up to some polynomial factor (see Section for an explanation of polynomial equivalence). Therefore, we don't actually care about which rules and axioms are used in our Frege system. This allows us to restrict our interest to a standardized Frege system, known in the literature as *Hilbert's propositional proof system*, denoted with F or Frege. The rule scheme of this proof system allows only the use of Modus Ponens over the following standard axiom scheme $\text{Ax}(F)$ of Hilbert-style systems for propositional logic:

add ref

add ref

- | | | |
|------|---|---------------------|
| (1) | $A \rightarrow B \rightarrow (A \wedge B)$ | |
| (2) | $(A \wedge B) \rightarrow A$ | $\wedge$ rules |
| (3) | $(A \wedge B) \rightarrow B$ | |
| (4) | $A \rightarrow (A \vee B)$ | $\vee$ rules |
| (5) | $B \rightarrow (A \vee B)$ | |
| (6) | $(A \rightarrow C) \rightarrow (B \rightarrow C) \rightarrow ((A \vee B) \rightarrow C)$ | |
| (7) | $(\perp \rightarrow A)$ | $\neg$ rules |
| (8) | $(A \rightarrow \perp) \rightarrow \neg A$ | |
| (9) | $A \rightarrow (\neg A \rightarrow \perp)$ | |
| (10) | $(\neg A \rightarrow \perp) \rightarrow A$ | |
| (11) | $A \rightarrow A$ | $\rightarrow$ rules |
| (12) | $A \rightarrow (B \rightarrow A)$ | |
| (13) | $(A \rightarrow (B \rightarrow C)) \rightarrow ((A \rightarrow B) \rightarrow (A \rightarrow C))$ | |

Given a formula A , an F-proof of A is sequence of formulas $\pi = (A_1, \dots, A_t)$ such that each $i \in [t]$ exactly one of the following holds:

- *Axiom*: $A_i \in \text{Ax}(\text{F})$;
- *Modus Ponens*: $A_i = \text{MP}(A_j, A_k)$ with $j, k \in [i - 1]$.

For the last formula A_t , it must hold that $A = A_t$. The soundness of F comes immediately from the soundness of Modus Ponens, something that assumed as an axiom in standard settings of classical Logic. Completeness follows similarly.

Extended Frege systems, denoted with EF, are Frege systems augmented with an *extension rule* that allows the introduction of new variables that abbreviate more complex formulas.

- *Extension*: $A_i = q \leftrightarrow B$ where q is a new variable that never appeared before and B is any propositional formula.

Intuitively, this rule allows us to write shorter proofs by encoding partial statements are single variables and use it in the proof, instead of carrying over the initial statement across the whole proof. In some sense, Frege proofs can be viewed as proofs that reason about polynomial size Boolean formulas, while Extended Frege proofs as proofs that reason about polynomial size Boolean circuits.

3.2.3 Algebraic systems

Algebraic proof systems are proof systems based on polynomials. These systems can be used to witness the fact that a system of polynomial equations over a the polynomial ring $\mathbb{F}[x_1, \dots, x_n]$ is unsolvable; equivalently, that the polynomials do not have a common root.

Consider a CNF formula φ defined over $x_1, \dots, x_n$. The formula can be easily encoded as a set of polynomials such that the formulas is unsatisfiable if and only if

such set of polynomials has no solution. We map each clause of φ to an equation in the following way:

$$\bigvee_{i \in I} x_i \vee \bigvee_{j \in J} \neg x_j \mapsto \prod_{i \in I} x_i \prod_{j \in J} (1 - x_j) = 0$$

where I and J are, respectively, the set of positive and negative literals of the clause. For each clause C , its corresponding polynomial is denoted with $P(C)$. The formula φ is unsatisfiable if and only if the system $\Phi = \{P(C)\}_{C \in \varphi} \cup \{x_i^2 - x_i = 0\}_{i \in [n]}$ has no solutions. In particular, the latter set of equations is used to restrict all possible solutions to those that give Boolean values to $x_1, \dots, x_n$.

After this transformation is made, the question of whether a formula is unsatisfiable is reduced to the question of a set of polynomial equations is unsolvable. Algebraic tools can then be used in order to decide unsatisfiability of the original formulas. We notice that, under this framework, truth is inversely represented: the value 0 represents truth, while 1 represents falsity.

Two particular algebraic proof systems are extensively studied in proof complexity: Nullstellensatz and Polynomial Calculus. *Nullstellensatz* (NS) is a static algebraic proof system, i.e. one that doesn't use rules of inference, based on Hilbert's Nullstellensatz, which states that given m polynomials $p_1, \dots, p_m$ the system $p_1(x) = p_2(x) = \dots = p_m(x) = 0$ is unsolvable if and only if 1 lies in their generated ideal, i.e. there are m polynomials $g_1, \dots, g_m$ such that $\sum_{j=1}^m g_j p_j = 1$. For any CNF φ , an NS-proof of φ is the set of polynomials $\pi = \{g_1, \dots, g_m, h_1, \dots, h_m\}$ such that:

$$\sum_{j=1}^m g_j p_j + \sum_{i=1}^n h_i (x_i^2 - x_i) = 1$$

where $p_j = P(C_j)$ for each $j \in [m]$.

Polynomial Calculus (PC) is a dynamic proof system based on two rules:

$$\frac{p}{\alpha p + \beta q} \quad (\text{linear combination}) \qquad \frac{p}{xp} \quad (\text{multiplication})$$

where $\alpha, \beta \in \mathbb{F}$ and $x \in \{x_1, \dots, x_n\}$. A PC-proof of the CNF φ is a sequence of polynomials $\pi = (f_1, \dots, f_t)$ such that each $i \in [t-1]$ exactly one of the following holds:

- *Axiom*: f_i is the polynomial of some equation in Φ
- *Linear combination*: $f_i = \text{Sum}(f_j, f_k)$ with $j, k \in [i-1]$
- *Multiplication*: $f_i = \text{Mul}(f_j)$ with $j \in [i-1]$

For the last polynomial f_t , it must hold that $f_t = 1$. The soundness of PC can be proven in way similar to Resolution: if we assume that the system of equations Π has a solution, the equality with 0 can be propagated down to the last polynomial $f_t = 1$, raising a contradiction. Completeness follows similarly.

3.2.4 Optimization-based systems

Optimization-based proof systems are proof systems based on linear programming. All such proof systems are variants of the *Cutting Planes* (CP) proof system, a refutational method derived from 0-1 integer programming. In CP, any CNF formula φ is converted into an equivalent system of linear inequalities (similar to how algebraic systems convert them into equations) whose integer binary solutions correspond directly to the satisfying assignments of φ . Each clause of a formula φ is mapped to an inequality in the following way:

$$\bigvee_{i \in I} x_i \vee \bigvee_{j \in J} \neg x_j \mapsto \sum_{i \in I} x_i + \sum_{j \in J} (1 - x_j) \geq 1$$

For each clause C , its corresponding inequality is denoted with $E(C)$. Interpreting truthness as 1 and falseness as 0, the formula φ is unsatisfiable if and only if the system $\Phi = \{E(C)\}_{C \in \varphi} \cup \{x_i \geq 0; x_i \leq 1\}_{i \in [n]}$ has no an integer solution. The set of all feasible solutions for this system yield a polytope P . While any Boolean point in P yields a satisfying assignment for F , an unsatisfiable formula F may still leave non-integer (fractional) points within P . CP eliminates these fractional solutions using the *Chvátal-Gomory cut rule*. In particular, CP is based on the two following rules:

$$\frac{\sum_{i \in [n]} \alpha_i x_i \geq \gamma \quad \sum_{i \in [n]} \beta_i x_i \geq \delta}{\sum_{i \in [n]} (\alpha_i + \beta_i) x_i \geq \gamma + \delta} \quad (\text{linear combination})$$

$$\frac{\sum_{i \in [n]} \eta \alpha_i x_i \geq \gamma}{\sum_{i \in [n]} \alpha_i x_i \geq \left\lfloor \frac{\gamma}{\eta} \right\rfloor} \quad (\text{cut})$$

Formally, a CP-proof of the CNF φ is a sequence of inequalities $\pi = (f_1, \dots, f_t)$ such that each $i \in [t - 1]$ exactly one of the following holds:

- *Axiom*: $f_i \in \Phi$
- *Linear combination*: $f_i = \text{Sum}(f_j, f_k)$ with $j, k \in [i - 1]$
- *Multiplication*: $f_i = \text{Cut}(f_j)$ with $j \in [i - 1]$

For the last inequality f_t , it must hold that $f_t : 0 \geq 1$ to ensure a contradiction.

3.2.5 Tree-like systems

Consider again the refutation in [Figure 1](#). To make things easier to read, we can represent this refutation as a Directed Acyclic Graph (DAG) by connecting clauses with lines. These refutations are called *DAG-like refutations*.

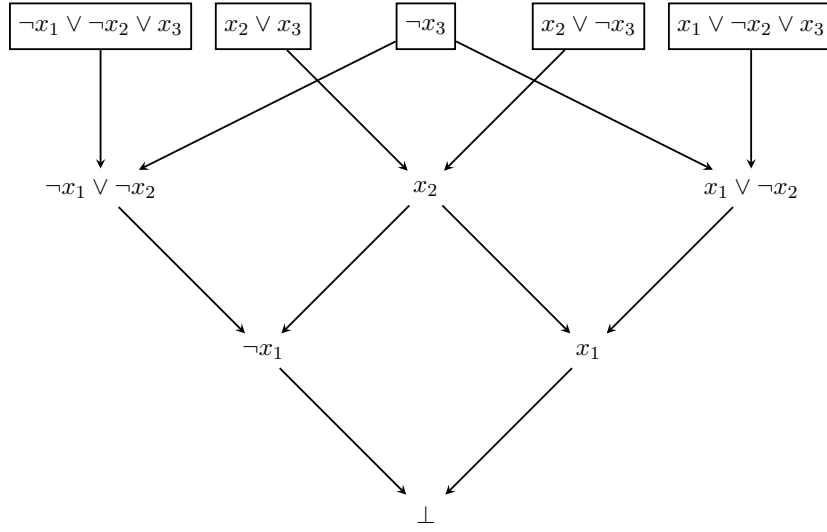


Figure 2. DAG-like representation of the refutation in Figure 1.

We observe that the definition of Resolution refutation allows rules to be applied more than once onto each clause of the proof. There may also be multiple copies of a single clause, each obtained by applying different rules. When each copy of a clause is used only once as premise in the whole proof, we say that the refutation is *tree-like* (named after the fact that the graph becomes a tree). This restriction implies that if we want to use a clause multiple times then it must be derived again, usually starting from the same axioms.

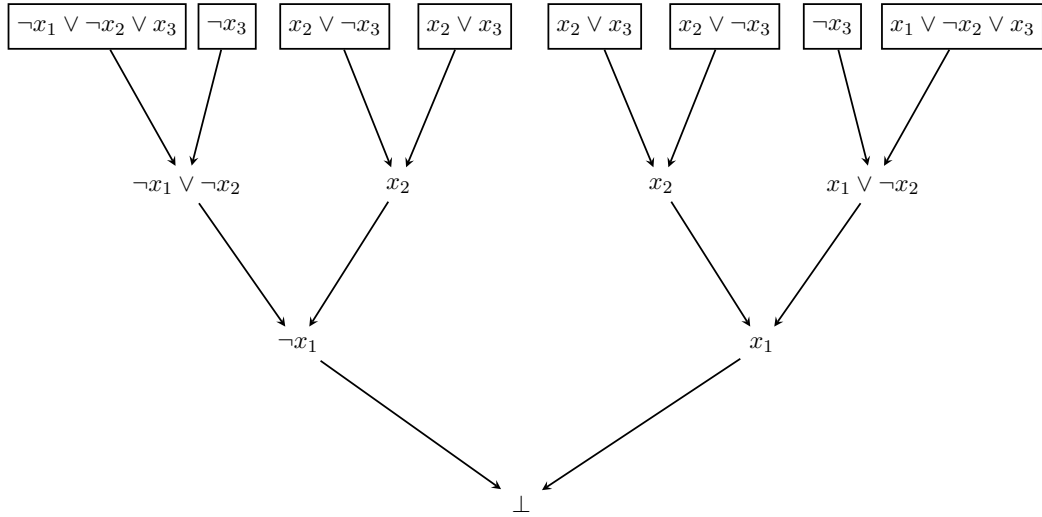


Figure 3. Tree-like adaptation of the refutation in Figure 1.

This sub-system of Resolution is called *tree-like Resolution*, denoted with Res^* . The same idea can also be applied to other dynamic systems, obtaining sub-systems like $\text{Res}^*(k)$, F^* and PC^* . Generally, tree-like sub-system require longer proofs due to the impossibility of reusing derived clauses.

3.3 Simulations between proof systems

To compare the relative strength of different propositional proof systems, we introduce the notion of efficient simulation. This notion comes in a natural way by adapting the idea of polynomial time reduction.

Definition 3.2. Given two proof systems P, Q , we say that Q polynomially simulates P , written $P \leq_p Q$ if there is a polynomial time computable function $f : \Sigma^* \rightarrow \Sigma^*$ such that for all tautologies φ if π is a P -proof of φ then $f(\pi)$ is a Q -proof of φ .

Intuitively, this means that there is an efficient algorithm transforming P -proofs into Q -proofs for the same formula. Consider the binary relation $\leq_p$. Similarly to reductions, this formula forms a partial order. In particular, for any pair P, Q of proof systems we say that:

- P and Q are *polynomially equivalent* when $P \leq_p Q$ and $Q \leq_p P$. We often write $P \equiv_p Q$ when this holds.
- P is *superpolynomially weaker* than Q when $P \leq_p Q$ but $Q \not\leq_p P$
- P and Q are *incomparable* when $P \not\leq_p Q$ and $Q \not\leq_p P$

It's easy to see that for any dynamic proof system Q it holds that $Q^* \leq_p Q$ since any Q^* -proof is also a Q -proof. Vice versa, however, it commonly holds that $Q \not\leq_p Q^*$, making tree-like systems typically weaker than their DAG-like counterpart. For instance, standard techniques can prove that the Ordering Principle formula OP_n is a hard formula for tree-like Resolution but an easy one for Resolution, implying that $\text{Res} \not\leq_p \text{Res}^*$.

When it comes to comparing Resolution with Nullstellensatz and Polynomial Calculus, they're both incomparable with it. This is due to the fact that negative literals easily blow up the size of the proof due to the increase in the number of monomials when expanded. To fix this, we can consider a variant of Polynomial Calculus referred to as *Polynomial Calculus with Resolution* (PCR) – a name that pretty much says it all – where for each variable x_i we add a new variable $\bar{x}_i$ in order to map negative literals $\neg x_i$ directly to this new variable.

$$\bigvee_{i \in I} x_i \vee \bigvee_{j \in J} \neg x_j \mapsto \prod_{i \in I} x_i \prod_{j \in J} \bar{x}_j = 0$$

All the other standard simulations and separations between proof systems are omitted and summarized in [Figure 4](#).

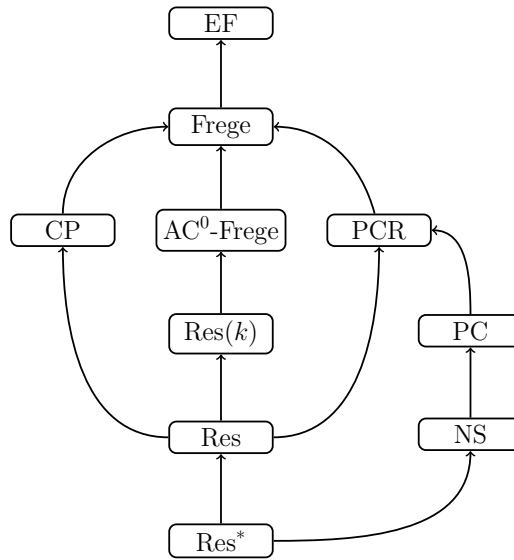


Figure 4. Simulations between standard proof systems.

add references for simulations (also inside the image)

3.4 Complexity measures for proof systems

Complexity measures lie at the heart of proof complexity. These measures provide a rigorous language to measure if a minimal proof must be short or long, allowing us to evaluate how different systems perform on proving a common statement. Different measures highlight different facets of a proof, often interlinking them. By analyzing these measures, we gain a deeper understanding of the inherent difficulty of logical reasoning, even from an algorithmic prospective.

Resolution is the quintessential proof system when it comes to complexity measures. Consider a Res-proof π . For any clause C , let $|C|$ denote the number of literals in C . Our main complexity measure of interest is The *size* of π , written as $|\pi|$, corresponding to the total number of literals inside it, i.e. $|\pi| = \sum_{D_i \in \pi} |D_i|$. For instance, the refutation in Figure 1 has size 19. In a natural way, the size of the proof corresponds to the number of symbols used to describe the witness of the verifier-based definition of the proof system. For any formula φ with m clauses over n variables, the Resolution size of φ , written as $\text{size}_{\text{Res}}(\varphi)$, is the minimum size across all Res-proofs of φ (equivalently, across all refutations of $\neg\varphi$).

$$\text{size}_{\text{Res}}(\varphi) = \min_{\pi: \text{Res} \vdash \varphi} |\pi|$$

Finding the Resolution size of a formula tells us the exact minimum number of symbols in a witnessing string for the given formula. Proving the existence of at least one formula φ , with m clauses over n variables, whose Resolution size is super-polynomial with respect to $|\varphi| = \Theta(n + m)$ is sufficient to conclude Resolution is not polynomially bounded. Haken was the first to show that one such formula is the Pigeonhole Principle formula PHP_n^{n+1} , which has size $2^{\Theta(n)}$. This result not only implies that Resolution is polynomially bounded, but it also implies that every CNF

add ref

formula that implicitly describes the Pigeonhole Principle is hard for Resolution. Since this combinatorial principle is at the foundation of mathematics, this makes Resolution weak against a wide subset of formulas.

A natural complexity measure related with size is the *length* of a proof, which measures the total number of clauses inside π , i.e. the number of steps in the proof. The Resolution length of a formula φ , written as $\text{length}_{\text{Res}}(\varphi)$, is the minimum length across all Res-proofs of φ . One can easily prove that $\text{length}_{\text{Res}}(\varphi) \leq \text{size}_{\text{Res}}(\varphi) \leq n \cdot \text{length}_{\text{Res}}(\varphi)$ holds for any φ . For this reason, the concepts of size and length are often treated as synonyms in the case of Resolution.

Another complexity measure inspired by Haken's proof is the *width* of a proof π , defined as the maximum number of literals across all clauses of π , i.e. $\max_{D_i \in \pi} |D_i|$. The Resolution width of φ , written as $\text{width}_{\text{Res}}(\varphi)$, is defined as the minimum across all proofs. Ben-Sasson and Wigderson proved that Resolution size and width are related one another through the trade-off.

add ref

Theorem 3.3. $\text{size}_{\text{Res}}(\varphi) = \exp \left(\Omega \left(\frac{(\text{width}_{\text{Res}}(\varphi) - \text{width}(F))^2}{n} \right) \right)$

We observe that in the exponent of the right hand side of the equation we have the difference between the Resolution width and the width of the axioms in φ . Because of this, for the case of families of formulas with large initial clauses, like the Pigeonhole Principle formula PHP_n^{n+1} , this method can prove only trivial size lower bounds (in fact, Haken's proof uses width of *monotone* clauses).

All Resolution measures carry over to Res^* , $\text{Res}(k)$ and CP, with some small variations: in $\text{Res}(k)$ the concept of width translates to the number of k -DNFs inside the conjunctions, while in CP there is no real concept of width. These adaptations imply that the above trade-off theorem doesn't have an analogous in these two worlds.

Moving to other proof systems, size and length measures are inherited by Frege systems in a natural way, but the relation between $\text{size}_{\text{Frege}}(\varphi)$ and $\text{length}_{\text{Frege}}(\varphi)$ and length is not so straightforward anymore since the formulas inside the proof are not restricted to being clauses. Nonetheless, Reckhow proved that they are still polynomially related one another.

add ref

Theorem 3.4. $\text{size}_{\text{Frege}}(\varphi) = O(\text{length}_{\text{Frege}}(\varphi) \cdot (|\varphi| + \text{length}_{\text{Frege}}(\varphi))^2)$

Other than this, no general trade-offs are known for Frege systems. In addition, there is no concept of width (again, formulas are not restricted to being clauses). Nonetheless, there is another complexity measures of our interest called *nested depth*, that being the maximum nesting level of logical connectives across all formulas of the proof. For general Frege system, there are no known formulas that require exponential size proofs. However, when restricted to $\Theta(1)$ nested depth, some formulas are known. This sub-system of Frege is known as AC^0 -Frege. Furthermore, it can be easily proven that Frege is equivalent to Resolution when restricted to depth-1 formulas.

Both in Nullstellensatz and Polynomial Calculus, size is defined differently. The size of a polynomial $||p||$ is defined as the number of non-zero monomials when expanded out as a linear combination of monomials. The size of a proof

$||\pi||_Q$, with $Q \in \{\text{NS}, \text{PC}\}$, is the sum of the sizes of all the polynomials inside it. Regarding polynomials, we have another complexity measure: *degree*. The degree of a polynomial $\deg(p)$ is the maximum degree of all monomials inside it. The degree of a proof is $\deg_Q(\pi)$ is the maximum degree across all polynomials inside it. The values of $\text{size}_Q(\varphi)$, $\deg_Q(\varphi)$ and are defined accordingly.

In particular, for a NS-proof $\pi = \{g_1, \dots, g_m, h_1, \dots, h_n\}$ over the system $\{p_1, \dots, p_m\} \cup \{x_i^2 - x_i\}_{i \in [n]}$ the size and degree of π are defined by:

$$||\pi||_{\text{NS}} = \sum_{j=1}^m ||g_j|| ||p_j|| + \sum_{i=1}^n 2 ||h_i||$$

$$\deg_{\text{NS}}(\pi) = \max_{j \in [m], i \in [n]} (\deg(g_j) + \deg(p_j), 2 + \deg(h_i))$$

For a PC-proof $\pi' = (f_1, \dots, f_t)$, instead, the size of π' is defined as $||\pi'||_{\text{PC}} = \sum_{i=1}^t ||f_i||$ and the degree is $\max_{i \in [t]} (\deg(f_i))$.

3.5 Game characterizations of Resolution measures

3.5.1 The Prover-Delayer Game (TODO)

3.5.2 The Spoiler-Duplicator Game (TODO)

Chapter 4

Automatability

Chapter 5

Analyzability

Conclusions

Acknowledgements

Bibliography

- [Pou92] William Poundstone. *Prisoner's Dilemma: John Von Neumann, Game Theory and the Puzzle of the Bomb*. Doubleday, 1992. ISBN: 978-0385415804.
- [Sip92] Michael Sipser. “The history and status of the P versus NP question”. In: *Proceedings of the Twenty-Fourth Annual ACM Symposium on Theory of Computing*. STOC '92. Association for Computing Machinery, 1992. DOI: 10.1145/129712.129771.
- [Tur37] A. M. Turing. “On Computable Numbers, with an Application to the Entscheidungsproblem”. In: *Proceedings of the London Mathematical Society* s2-42.1 (Jan. 1937), pp. 230–265. ISSN: 0024-6115. DOI: 10.1112/plms/s2-42.1.230.